

How the model works — two models, one product

We rank each month's new donors by **expected dollars**, because the whole project is scored in dollars per contact.

In plain words

- 1 A **decision tree** is a flowchart of questions: "Gift over \$80? A Music project? In Mississippi?" — then a guess.
- 2 **Boosting** builds hundreds of tiny trees in sequence — each new tree only fixes what the previous ones still get wrong. Like revising a memo 300 times, one small fix per pass.
- 3 We build **two**: Model A asks *"will this donor give again?"* (a probability). Model B asks *"if they do, how big is the gift?"* — learned only from donors who came back.
- 4 Multiply and rank: **18% × \$65 ≈ \$11.70** of expected future giving. The development lead calls from the top of that list.

The technical spec

Score. $P(\text{return}) \times E[\text{amount} \mid \text{return}]$ — the amount side is a GB regression on $\log(\text{amount})$, fit on returners only (Albert's approved approach).

Algorithm. HistGradientBoosting (scikit-learn) — LightGBM's algorithm, zero new dependencies; native categoricals and missing values.

Features. First-gift attributes + the projects join (subject, grade, state, cost, free-lunch %). Trees learn the state × gift-size crosses — where Reid found the signal — automatically.

Leakage rules. Excluded: thank-you packet (no timing in codebook), teacher IDs, all post-outcome fields.

Validation. Fit ≤ 2015 , validate on 2016 (time split, mirrors the real one). Holdout 2017–18 gated behind a flag, scored exactly once. Early stopping, L2, bootstrap SE next to every number.